



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 6

Team No.: 9

Project Title: Linux Inter-Process Communication Using Pipes and FIFOs

Team Members

Roll Number	Student Name	Signature
2520030431	Varshini Bolla	
2520030436	Eesha Yuktha	
2520030556	Hamsini Vanam	

1. Abstract

Inter-Process Communication (IPC) is an important concept in Operating Systems that allows different processes to exchange data and coordinate their activities. In Linux systems, processes normally have separate memory spaces, so a mechanism is required for safe and efficient communication between them. This project, “**Linux Inter-Process Communication Using Pipes and FIFOs,**” focuses on understanding and implementing two basic IPC mechanisms: pipes and named pipes (FIFOs).

The project demonstrates how a Linux-based system can enable communication between processes using system calls and file descriptors. Pipes provide a simple way to transfer data between related processes, such as a parent and child process, while FIFOs allow communication between unrelated processes through a special named file. The project covers relevant Operating Systems and System Programming concepts such as process creation, process synchronization, file descriptors, system calls, reading and writing data, and process termination.

The proposed solution will be implemented in C programming on Linux. Sample programs will be developed to create processes and demonstrate data transfer using both unnamed pipes and FIFOs. The communication flow and behavior of the processes will be observed and compared.

The expected outcome of the project is a clear practical understanding of IPC in Linux and how pipes and FIFOs can be used for process communication. It will also help students gain hands-on experience with Linux system calls, process management, and basic system programming.

2. Problem Statement

In a multitasking operating system like Linux, multiple processes are executed independently and often need to exchange information or coordinate their activities. Since each process has its own memory space, one process cannot directly access the memory of another process. Therefore, an efficient Inter-Process Communication (IPC) mechanism is required for processes to safely exchange data.

This project addresses the problem of how processes can communicate and transfer data in a Linux environment using pipes and FIFOs. It focuses on understanding the difference between unnamed pipes, which are commonly used between related processes, and named pipes (FIFOs), which can be used for communication between unrelated processes.

The project is relevant to Operating Systems because IPC is an important part of process management and synchronization. It is also relevant to Systems Programming because implementing IPC involves Linux system calls, file descriptors, process creation, and input/output operations.

3. Objectives

(List the specific objectives of the project.)

1. To understand the concept of Inter-Process Communication (IPC) and its importance in Linux Operating Systems.
2. To implement communication between processes using unnamed pipes in C programming.
3. To implement and demonstrate communication between unrelated processes using named pipes (FIFOs).
4. To analyze and compare the working of pipes and FIFOs and gain practical experience with Linux system calls and process management.

4. Proposed Methodology

1. The project will be designed and implemented in a Linux environment using the C programming language.
2. Processes will be created using Linux system calls such as `fork()` to demonstrate communication between related processes.
3. An unnamed pipe will be created using the `pipe()` system call. One process will write data into the pipe, while another process will read and display the data.
4. A named pipe (FIFO) will be created using the `mkfifo()` system call to demonstrate communication between unrelated processes.
5. System calls such as `read()`, `write()`, and `close()` will be used to transfer data and manage communication channels.
6. Different input messages will be tested to verify that data is successfully transferred between processes.
7. The working of pipes and FIFOs will be observed and compared based on their usage, communication process, and suitability for different types of processes.
8. The overall flow of the project will be: Create Processes → Create IPC Channel → Write Data → Read Data → Display Output → Close Communication Channel → Terminate Processes.
9. The project will provide practical understanding of Linux process management, system calls, file descriptors, and Inter-Process Communication.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
--------------------------------------	------------------------

Process Creation – fork()	Creates a child process to demonstrate communication between related processes.
Unnamed Pipe – pipe()	Creates a communication channel for transferring data between related processes.
Named Pipe (FIFO) – mkfifo()	Creates a named communication channel for communication between unrelated processes.
read() and write()	Used to receive and send data through pipes and FIFOs.
File Descriptors and close()	Used to manage communication channels and close unused pipe/FIFO ends.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030431	Varshini Bolla	Implement communication between processes using named pipes (FIFOs) and mkfifo().
2520030436	Eesha Yuktha	Study IPC concepts and implement communication using unnamed pipes and fork().
2520030556	Hamsini Vanam	Perform testing and comparison of pipes and FIFOs, and prepare documentation and presentation.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system used to develop, compile, and execute the project.
C Programming Language	Used to implement the IPC programs using Linux system calls.
GCC Compiler	Used to compile the C programs and generate executable files.
Linux Terminal	Used to compile the C programs and generate executable files.
Visual Studio Code	Used as a code editor for writing and managing the C source code.

8. Expected Outcome

The project is expected to successfully demonstrate Inter-Process Communication (IPC) in a Linux environment using pipes and FIFOs. The unnamed pipe implementation will allow a parent and child to process data, while the FIFO implementation will allow two independent processes to communicate through a named pipe. The programs will display the data sent by one process and received by another process on the Linux terminal. The project will also demonstrate the practical use of Linux system calls such as fork(), pipe(), mkfifo(), read(), write(), and close(). Overall, the project will help students understand the difference between pipes and FIFOs and gain practical knowledge of process management, IPC, Linux system calls, and basic system programming.

9. GitHub Repository

GitHub Repository Link: <https://github.com/2520030431-436-556-OSSP-PROJECT>

Repository Created: Yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Dr. K. Anusha

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____